Compare

Compare

A 3-file React component from Aceternity UI. Add it to this project so it works and looks exactly like the source below.

Fastest path

If this project has a `components.json` (shadcn), run `npx shadcn@latest add @aceternity/compare-demo`.
If the command succeeds, only fix import paths to match this project and skip to **Check**. If it fails or there is no `components.json`, build it by hand as below.

Build it

- Stack: React with TypeScript and Tailwind CSS v4.
- Install: `motion`, `@tabler/icons-react`, `@tsparticles/react`, `@tsparticles/engine`, `@tsparticles/slim`, `clsx`, `tailwind-merge`. Use this project's package manager.
- Create these files, in this project's usual place for components:
  - `components/ui/compare.tsx`
  - `components/ui/sparkles.tsx`
- Keep these exports and their names: `Compare`, `SparklesCore`. Other code imports them.
- It imports `cn` from `@/lib/utils` (clsx + tailwind-merge, so a caller's `className` overrides the defaults). Use the project's helper if it has one; otherwise create it: `export function cn(...inputs: ClassValue[]) { return twMerge(clsx(inputs)); }`.
- Imports that start with `@/` use the shadcn alias. Rewrite them to this project's alias, and keep the file layout consistent with its conventions (location, naming, formatting).
- Keep `"use client"` at the top in Next.js App Router projects (it uses state, effects or browser APIs). In other setups it is harmless.
- Style with the Tailwind utilities exactly as written. They are the design: do not swap them for other values, inline styles or a CSS file.
- Animation uses Motion. Keep the timings, easings and the reduced-motion handling as they are.
- Images load from `assets.aceternity.com`. Keep them for now, or replace them with this project's own images; in Next.js, allow that host if `next/image` is used.

Use it

The example below shows how to render it. Put it on a page to see it working:
- `components/compare-demo.tsx`

Check

- It type-checks and the page renders with no console errors.
- It looks and behaves like the source in light and dark mode.
- With `prefers-reduced-motion: reduce`, motion is reduced as the source intends.
- Tell me every place where you had to do something differently from the source, and why.

Source

The reference implementation. Copy it as-is, then apply only the changes listed above.

`components/compare-demo.tsx`

```tsx
import React from "react";
import { Compare } from "@/components/ui/compare";

export default function CompareDemo() {
  return (
    <div classNam="p-4 border rounded-3xl dark:bg-neutral-900 bg-neutral-100  border-neutral-200 dark:border-neutral-800 px-4">
      <Compare
        firstImage="https://assets.aceternity.com/code-problem.png"
        secondImage="https://assets.aceternity.com/code-solution.png"
        firstImageClassName="object-cover object-left-top"
```

```
      secondImageClassname="object-cover object-left-top"
      className="h-[250px] w-[200px] md:h-[500px] md:w-[500px]"
      slideMode="hover"
    />
   </div>
 );
}
```

`components/ui/compare.tsx`

```tsx
"use client";
import React, { useState, useEffect, useRef, useCallback } from "react";
import { SparklesCore } from "@/components/ui/sparkles";
import { AnimatePresence, motion } from "motion/react";
import { cn } from "@/lib/utils";
import { IconDotsVertical } from "@tabler/icons-react";

interface CompareProps {
 firstImage?: string;
 secondImage?: string;
 className?: string;
 firstImageClassName?: string;
 secondImageClassname?: string;
 initialSliderPercentage?: number;
 slideMode?: "hover" | "drag";
 showHandlebar?: boolean;
 autoplay?: boolean;
 autoplayDuration?: number;
}
export const Compare = ({
 firstImage = "",
 secondImage = "",
 className,
 firstImageClassName,
 secondImageClassname,
 initialSliderPercentage = 50,
 slideMode = "hover",
 showHandlebar = true,
 autoplay = false,
 autoplayDuration = 5000,
}: CompareProps) => {
 const [sliderXPercent, setSliderXPercent] = useState(initialSliderPercentage);
 const [isDragging, setIsDragging] = useState(false);

 const sliderRef = useRef<HTMLDivElement>(null);

 const [isMouseOver, setIsMouseOver] = useState(false);

 const autoplayRef = useRef<NodeJS.Timeout | null>(null);

 const startAutoplay = useCallback(() => {
  if (!autoplay) return;

  const startTime = Date.now();
  const animate = () => {
   const elapsedTime = Date.now() - startTime;
   const progress =
```

```
      (elapsedTime % (autoplayDuration * 2)) / autoplayDuration;
    const percentage = progress <= 1 ? progress * 100 : (2 - progress) * 100;

    setSliderXPercent(percentage);
    autoplayRef.current = setTimeout(animate, 16); // ~60fps
  };

  animate();
}, [autoplay, autoplayDuration]);

const stopAutoplay = useCallback(() => {
  if (autoplayRef.current) {
    clearTimeout(autoplayRef.current);
    autoplayRef.current = null;
  }
}, []);

useEffect(() => {
  startAutoplay();
  return () => stopAutoplay();
}, [startAutoplay, stopAutoplay]);

function mouseEnterHandler() {
  setIsMouseOver(true);
  stopAutoplay();
}

function mouseLeaveHandler() {
  setIsMouseOver(false);
  if (slideMode === "hover") {
    setSliderXPercent(initialSliderPercentage);
  }
  if (slideMode === "drag") {
    setIsDragging(false);
  }
  startAutoplay();
}

const handleStart = useCallback(
  (clientX: number) => {
    if (slideMode === "drag") {
      setIsDragging(true);
    }
  },
  [slideMode]
);

const handleEnd = useCallback(() => {
  if (slideMode === "drag") {
    setIsDragging(false);
  }
}, [slideMode]);

const handleMove = useCallback(
  (clientX: number) => {
    if (!sliderRef.current) return;
    if (slideMode === "hover" || (slideMode === "drag" && isDragging)) {
      const rect = sliderRef.current.getBoundingClientRect();
      const x = clientX - rect.left;
      const percent = (x / rect.width) * 100;
```

```
      requestAnimationFrame(() => {
        setSliderXPercent(Math.max(0, Math.min(100, percent)));
      });
    }
  },
  [slideMode, isDragging]
);

const handleMouseDown = useCallback(
  (e: React.MouseEvent) => handleStart(e.clientX),
  [handleStart]
);
const handleMouseUp = useCallback(() => handleEnd(), [handleEnd]);
const handleMouseMove = useCallback(
  (e: React.MouseEvent) => handleMove(e.clientX),
  [handleMove]
);

const handleTouchStart = useCallback(
  (e: React.TouchEvent) => {
    if (!autoplay) {
      handleStart(e.touches[0].clientX);
    }
  },
  [handleStart, autoplay]
);

const handleTouchEnd = useCallback(() => {
  if (!autoplay) {
    handleEnd();
  }
}, [handleEnd, autoplay]);

const handleTouchMove = useCallback(
  (e: React.TouchEvent) => {
    if (!autoplay) {
      handleMove(e.touches[0].clientX);
    }
  },
  [handleMove, autoplay]
);

return (
  <div
    ref={sliderRef}
    className={cn("w-[400px] h-[400px] overflow-hidden", className)}
    style={{
      position: "relative",
      cursor: slideMode === "drag" ? "grab" : "col-resize",
    }}
    onMouseMove={handleMouseMove}
    onMouseLeave={mouseLeaveHandler}
    onMouseEnter={mouseEnterHandler}
    onMouseDown={handleMouseDown}
    onMouseUp={handleMouseUp}
    onTouchStart={handleTouchStart}
    onTouchEnd={handleTouchEnd}
    onTouchMove={handleTouchMove}
```

```
  >
    <AnimatePresence initial={false}>
      <motion.div
        className="h-full w-px absolute top-0 m-auto z-30 bg-gradient-to-b from-transparent from-[5%] to-[95%] via-indigo-500 to-transparent"
        style={{
          left: `${sliderXPercent}%`,
          top: "0",
          zIndex: 40,
        }}
        transition={{ duration: 0 }}
      >
        <div className="w-36 h-full [mask-image:radial-gradient(100px_at_left,white,transparent)] absolute top-1/2 -translate-y-1/2 left-0
bg-gradient-to-r from-indigo-400 via-transparent to-transparent z-20 opacity-50" />
        <div className="w-10 h-1/2 [mask-image:radial-gradient(50px_at_left,white,transparent)] absolute top-1/2 -translate-y-1/2 left-0
bg-gradient-to-r from-cyan-400 via-transparent to-transparent z-10 opacity-100" />
        <div className="w-10 h-3/4 top-1/2 -translate-y-1/2 absolute -right-10 [mask-image:radial-gradient(100px_at_left,white,transparent)]">
          <MemoizedSparklesCore
            background="transparent"
            minSize={0.4}
            maxSize={1}
            particleDensity={1200}
            className="w-full h-full"
            particleColor="#FFFFFF"
          />
        </div>
        {showHandlebar && (
          <div className="h-5 w-5 rounded-md top-1/2 -translate-y-1/2 bg-white z-30 -right-2.5 absolute   flex items-center justify-center
shadow-[0px_-1px_0px_0px_#FFFFFF40]">
            <IconDotsVertical className="h-4 w-4 text-black" />
          </div>
        )}
      </motion.div>
    </AnimatePresence>
    <div className="overflow-hidden w-full h-full relative z-20 pointer-events-none">
      <AnimatePresence initial={false}>
        {firstImage ? (
          <motion.div
            className={cn(
              "absolute inset-0 z-20 rounded-2xl shrink-0 w-full h-full select-none overflow-hidden",
              firstImageClassName
            )}
            style={{
              clipPath: `inset(0 ${100 - sliderXPercent}% 0 0)`,
            }}
            transition={{ duration: 0 }}
          >
            <img
              alt="first image"
              src={firstImage}
              className={cn(
                "absolute inset-0  z-20 rounded-2xl shrink-0 w-full h-full select-none",
                firstImageClassName
              )}
              draggable={false}
            />
          </motion.div>
        ) : null}
```

```
      </AnimatePresence>
    </div>

    <AnimatePresence initial={false}>
     {secondImage ? (
      <motion.img
       className={cn(
        "absolute top-0 left-0 z-[19]  rounded-2xl w-full h-full select-none",
        secondImageClassname
       )}
       alt="second image"
       src={secondImage}
       draggable={false}
      />
     ) : null}
    </AnimatePresence>
   </div>
 );
};

const MemoizedSparklesCore = React.memo(SparklesCore);
```

`components/ui/sparkles.tsx`

```tsx
"use client";
import React, { useId, useMemo } from "react";
import { useEffect, useState } from "react";
import Particles, { initParticlesEngine } from "@tsparticles/react";
import type { Container, SingleOrMultiple } from "@tsparticles/engine";
import { loadSlim } from "@tsparticles/slim";
import { cn } from "@/lib/utils";
import { motion, useAnimation } from "motion/react";

type ParticlesProps = {
 id?: string;
 className?: string;
 background?: string;
 particleSize?: number;
 minSize?: number;
 maxSize?: number;
 speed?: number;
 particleColor?: string;
 particleDensity?: number;
};
export const SparklesCore = (props: ParticlesProps) => {
 const {
  id,
  className,
  background,
  minSize,
  maxSize,
  speed,
  particleColor,
  particleDensity,
 } = props;
 const [init, setInit] = useState(false);
```

```
useEffect(() => {
  initParticlesEngine(async (engine) => {
    await loadSlim(engine);
  }).then(() => {
    setInit(true);
  });
}, []);
const controls = useAnimation();

const particlesLoaded = async (container?: Container) => {
  if (container) {
    controls.start({
      opacity: 1,
      transition: {
        duration: 1,
      },
    });
  }
};

const generatedId = useId();
return (
  <motion.div animate={controls} className={cn("opacity-0", className)}>
    {init && (
      <Particles
        id={id || generatedId}
        className={cn("h-full w-full")}
        particlesLoaded={particlesLoaded}
        options={{
          background: {
            color: {
              value: background || "#0d47a1",
            },
          },
          fullScreen: {
            enable: false,
            zIndex: 1,
          },

          fpsLimit: 120,
          interactivity: {
            events: {
              onClick: {
                enable: true,
                mode: "push",
              },
              onHover: {
                enable: false,
                mode: "repulse",
              },
              resize: true as any,
            },
            modes: {
              push: {
                quantity: 4,
              },
              repulse: {
                distance: 200,
```

```
      duration: 0.4,
    },
  },
},
particles: {
  bounce: {
    horizontal: {
      value: 1,
    },
    vertical: {
      value: 1,
    },
  },
  collisions: {
    absorb: {
      speed: 2,
    },
    bounce: {
      horizontal: {
        value: 1,
      },
      vertical: {
        value: 1,
      },
    },
    enable: false,
    maxSpeed: 50,
    mode: "bounce",
    overlap: {
      enable: true,
      retries: 0,
    },
  },
  color: {
    value: particleColor || "#ffffff",
    animation: {
      h: {
        count: 0,
        enable: false,
        speed: 1,
        decay: 0,
        delay: 0,
        sync: true,
        offset: 0,
      },
      s: {
        count: 0,
        enable: false,
        speed: 1,
        decay: 0,
        delay: 0,
        sync: true,
        offset: 0,
      },
      l: {
        count: 0,
        enable: false,
```

```
      speed: 1,
      decay: 0,
      delay: 0,
      sync: true,
      offset: 0,
    },
  },
},
effect: {
  close: true,
  fill: true,
  options: {},
  type: {} as SingleOrMultiple<string> | undefined,
},
groups: {},
move: {
  angle: {
    offset: 0,
    value: 90,
  },
  attract: {
    distance: 200,
    enable: false,
    rotate: {
      x: 3000,
      y: 3000,
    },
  },
  center: {
    x: 50,
    y: 50,
    mode: "percent",
    radius: 0,
  },
  decay: 0,
  distance: {},
  direction: "none",
  drift: 0,
  enable: true,
  gravity: {
    acceleration: 9.81,
    enable: false,
    inverse: false,
    maxSpeed: 50,
  },
  path: {
    clamp: true,
    delay: {
      value: 0,
    },
    enable: false,
    options: {},
  },
  outModes: {
    default: "out",
  },
  random: false,
```

```
  size: false,
  speed: {
    min: 0.1,
    max: 1,
  },
  spin: {
    acceleration: 0,
    enable: false,
  },
  straight: false,
  trail: {
    enable: false,
    length: 10,
    fill: {},
  },
  vibrate: false,
  warp: false,
},
number: {
  density: {
    enable: true,
    width: 400,
    height: 400,
  },
  limit: {
    mode: "delete",
    value: 0,
  },
  value: particleDensity || 120,
},
opacity: {
  value: {
    min: 0.1,
    max: 1,
  },
  animation: {
    count: 0,
    enable: true,
    speed: speed || 4,
    decay: 0,
    delay: 0,
    sync: false,
    mode: "auto",
    startValue: "random",
    destroy: "none",
  },
},
reduceDuplicates: false,
shadow: {
  blur: 0,
  color: {
    value: "#000",
  },
  enable: false,
  offset: {
    x: 0,
    y: 0,
```

```
  },
},
shape: {
  close: true,
  fill: true,
  options: {},
  type: "circle",
},
size: {
  value: {
    min: minSize || 1,
    max: maxSize || 3,
  },
  animation: {
    count: 0,
    enable: false,
    speed: 5,
    decay: 0,
    delay: 0,
    sync: false,
    mode: "auto",
    startValue: "random",
    destroy: "none",
  },
},
stroke: {
  width: 0,
},
zIndex: {
  value: 0,
  opacityRate: 1,
  sizeRate: 1,
  velocityRate: 1,
},
destroy: {
  bounds: {},
  mode: "none",
  split: {
    count: 1,
    factor: {
      value: 3,
    },
    rate: {
      value: {
        min: 4,
        max: 9,
      },
    },
    sizeOffset: true,
  },
},
roll: {
  darken: {
    enable: false,
    value: 0,
  },
  enable: false,
```

```
  enlighten: {
    enable: false,
    value: 0,
  },
  mode: "vertical",
  speed: 25,
},
tilt: {
  value: 0,
  animation: {
    enable: false,
    speed: 0,
    decay: 0,
    sync: false,
  },
  direction: "clockwise",
  enable: false,
},
twinkle: {
  lines: {
    enable: false,
    frequency: 0.05,
    opacity: 1,
  },
  particles: {
    enable: false,
    frequency: 0.05,
    opacity: 1,
  },
},
wobble: {
  distance: 5,
  enable: false,
  speed: {
    angle: 50,
```